

Technical Review Report

PSX Portfolio Tracker Application

1. Executive Summary

This document provides a comprehensive technical review of the **PSX (Pakistan Stock Exchange) Portfolio Tracker**, a bespoke single-page web application developed using standard web technologies (HTML5, CSS3, and Vanilla JavaScript). The application serves as a foundational ledger for retail investors to track their holdings, calculate average entry prices, and manage their portfolio in real-time within the browser.

Objective of the Review

To analyze the current architecture, evaluate the implementation of business logic (specifically the averaging down mechanics), assess code quality, and provide actionable recommendations for scaling the application into a robust, production-ready tool.

2. Architectural Analysis

The application follows a classic client-side architecture without dependencies on external frameworks like React, Angular, or Vue. This choice ensures a lightweight footprint and extremely fast execution times.

Component	File	Role & Implementation Details
Structure	<code>index.html</code>	Defines a semantic layout utilizing distinct sections for the top navigation bar, the dynamic portfolio container, and floating action buttons.
Styling	<code>style.css</code>	Implements a responsive, clean UI. Uses CSS Grid for the header alignment and a fixed positioning strategy for the action buttons. The color palette correctly mimics institutional financial tools using 'Pakistan Green' (#115740).
Logic	<code>index.js</code>	Encapsulates the state management, business logic (Stock Position classes), DOM manipulation, and input validation processes.

3. Feature Assessment

3.1. Object-Oriented Data Structure

The implementation of the `StockPosition` class represents a strong architectural decision. By encapsulating data (ticker, shares, buy price) and methods (`getMarketValue()` , `averageDown()`) within a class, the code remains highly modular. This makes future expansion—such as adding dividend tracking or target sell prices—significantly easier compared to utilizing loose objects or flat arrays.

3.2. Cost Basis and Averaging Logic

A critical component of any portfolio tracker is correctly calculating the weighted average cost when an investor buys more shares of an existing holding at a different price. The logic implemented in the `averageDown(newShares, newPrice)` method is mathematically sound:

Formula utilized:

New Average Price = ((Current Shares × Current Price) + (New Shares × New Price)) / (Current Shares + New Shares)

This avoids the common pitfall of simply averaging the two price points without weighting them by the number of shares.

3.3. Dynamic DOM Rendering

The `updateDisplay()` function efficiently manages the state-to-UI synchronization. It intelligently clears the existing container (`innerHTML = ""`) and rebuilds the DOM elements based on the current state of the `portfolio` array. Furthermore, the use of `toLocaleString()` ensures that currency values are formatted correctly with commas and fixed decimal places, enhancing readability.

4. Identified Areas for Improvement (Code Quality & UX)

While the core engine is robust, there are several limitations inherent to the current iteration of the software that should be addressed in subsequent versions:

- **Volatility of Data (No Persistence):** Because the `portfolio` array resides entirely in the browser's runtime memory, all data is irrevocably lost if the user refreshes the page or closes the tab.
- **Blocking User Interface:** The use of native `window.prompt` and `window.alert` functions halts JavaScript execution and blocks browser interaction. While functional, they provide a suboptimal user experience as they cannot be styled to match the application's aesthetic.

- **Unrealized Profit/Loss Capability:** The current "Total Value" reflects the *Total Cost of Investment*. Without a mechanism to input or fetch the *Current Market Price* of a stock, the user cannot determine if their portfolio is operating at a gain or a loss.

5. Recommendations for Version 2.0

To transition this application from a basic ledger to a highly functional personal finance tool, the following roadmap is proposed:

Phase 1: Implement Local Storage

Integrate the browser's Web Storage API (`localStorage`). The application should serialize the `portfolio` array into a JSON string using `JSON.stringify()` whenever a stock is added, updated, or deleted. On page load, it should parse this string back into `StockPosition` instances.

Phase 2: Custom Modals for Data Entry

Deprecate `window.prompt()` in favor of hidden HTML forms that overlay the screen (modals). This will allow for better input validation, styled error messages, and a more professional feel.

Phase 3: Live Market Data API Integration

Integrate a financial data API to pull real-time or end-of-day PSX stock prices. This will automate the "Current Value" calculation, allowing the application to display real-time green (profit) or red (loss) indicators for each holding.

Generated on: June 8, 2026 | **Subject:** Codebase Analysis

This technical report outlines the current state and future roadmap of the provided web application.